

# End-to-End Stochastic Model Predictive dispatch for ACOPF via Generative Models

Jie Zhu, *Graduate Student Member, IEEE*, and Yinliang Xu, *Senior Member, IEEE*,

**Abstract**—This paper proposes. We also open source our method and implementation details at <https://github.com/JieZhu6/code>.

**Index Terms**—.

## I. INTRODUCTION

**T**HE Model Predictive dispatch (MPC) has gained prominence as a highly effective dispatch framework for managing DERs due to its inherent ability to handle operational constraints, predict future behavior, and optimize economic objectives over a receding horizon.

In intraday dispatch, model predictive dispatch (MPC) is a practical framework for rolling decision-making, as it updates dispatch actions according to the latest forecast information. To capture the uncertainties of renewable generation and load demand, stochastic MPC (SMPC) adopts a scenario-based formulation that is well suited to nonconvex operational models. Unlike robust optimization and distributionally robust optimization, which often require tractable reformulations that are difficult to obtain under nonconvex constraints, SMPC directly incorporates multiple uncertainty scenarios into the optimization problem.

However, SMPC remains computationally challenging for intraday operation, since the consideration of multiple uncertainty scenarios leads to large-scale nonconvex optimization problems. Using neural network (NN) as surrogate optimization solver provides a promising pathway toward efficient end-to-end solutions for SMPC problems with nonconvex power flow constraints. Nevertheless, a critical challenge is to ensure that the NN outputs satisfy the underlying physical and operational constraints.

Generative models used in the power system [1], [2], in AI conference [3]–[5]

## II. PROBLEM FORMULATION

Consider a transmission network with bus set  $\mathcal{N}$ , conventional-generator bus set  $\mathcal{N}_g$ , and reference bus  $r_0$ . Let  $\mathcal{N}_{PV}$  denotes the sets of P-V buses, with  $r_0$  excluded from  $\mathcal{N}_{PV}$ . An SMPC scheme is considered over the prediction

horizon  $\mathcal{T} = \{1, \dots, T\}$  with scenario set  $\mathcal{N}_s$ . The resulting SMPC-ACOPF problem is formulated as

$$\min \sum_{t \in \mathcal{T}} \sum_{i \in \mathcal{N}_{PV}} \left[ a_i (P_{i,t}^g)^2 + b_i P_{i,t}^g + c_i \right] + \frac{1}{|\mathcal{N}_s|} \sum_{s \in \mathcal{N}_s} \sum_{t \in \mathcal{T}} \left[ a_i (P_{s,r_0,t}^g)^2 + b_i P_{s,r_0,t}^g + c_{r_0} \right] \quad (1a)$$

$$\text{s.t. } P_{s,i,t}^g - P_{s,i,t}^d = V_{s,i,t} \sum_{j \in \mathcal{N}} V_{s,j,t} \quad (1b)$$

$$\times (G_{ij} \cos \theta_{s,ij,t} + B_{ij} \sin \theta_{s,ij,t}), \quad \forall s \in \mathcal{N}_s$$

$$Q_{s,i,t}^g - Q_{s,i,t}^d = V_{s,i,t} \sum_{j \in \mathcal{N}} V_{s,j,t} \quad (1c)$$

$$\times (G_{ij} \sin \theta_{s,ij,t} - B_{ij} \cos \theta_{s,ij,t}), \quad \forall s \in \mathcal{N}_s$$

$$P_{s,ij,t} = -G_{ij} V_{s,i,t}^2 + V_{s,i,t} V_{s,j,t} \quad (1d)$$

$$\times (G_{ij} \cos \theta_{s,ij,t} + B_{ij} \sin \theta_{s,ij,t}), \quad \forall s \in \mathcal{N}_s$$

$$Q_{s,ij,t} = B_{ij} V_{s,i,t}^2 + V_{s,i,t} V_{s,j,t} \quad (1e)$$

$$\times (G_{ij} \sin \theta_{s,ij,t} - B_{ij} \cos \theta_{s,ij,t}), \quad \forall s \in \mathcal{N}_s$$

$$|S_{s,ij,t}|^2 = P_{s,ij,t}^2 + Q_{s,ij,t}^2, \quad \forall s \in \mathcal{N}_s \quad (1f)$$

$$|S_{s,ij,t}| \leq \bar{S}_{ij}, \quad \forall s \in \mathcal{N}_s \quad (1g)$$

$$\underline{V}_i \leq V_{s,i,t} \leq \bar{V}_i, \quad \theta_{ij} \leq \theta_{s,ij,t} \leq \bar{\theta}_{ij}, \quad \forall s \in \mathcal{N}_s \quad (1h)$$

$$\underline{P}_i^g \leq P_{s,i,t}^g \leq \bar{P}_i^g, \quad \underline{Q}_i^g \leq Q_{s,i,t}^g \leq \bar{Q}_i^g, \quad \forall s \in \mathcal{N}_s \quad (1i)$$

$$-R_i^{\text{down}} \leq P_{s,i,t}^g - P_{s,i,t-1}^g \leq R_i^{\text{up}}, \quad \forall s \in \mathcal{N}_s \quad (1j)$$

$$\theta_{s,r_0,t} = 0, \quad \forall s \in \mathcal{N}_s. \quad (1k)$$

where  $P_{s,i,t}^d$  and  $Q_{s,i,t}^d$  denote the active and reactive load demands at bus  $i$  and time  $t$  under scenario  $s$ , respectively. The variables  $V_{s,i,t}$  and  $\theta_{s,i,t}$  denote the corresponding voltage magnitude and phase angle, with  $\theta_{s,ij,t} = \theta_{s,i,t} - \theta_{s,j,t}$ . For each P-V bus  $i \in \mathcal{N}_{PV}$ , the active-power dispatch  $P_{i,t}^g$  and voltage-magnitude set point  $V_{i,t}$  are shared across all scenarios and therefore carry no scenario index. The variables  $P_{s,ij,t}$ ,  $Q_{s,ij,t}$ , and  $S_{s,ij,t}$  denote the active, reactive, and apparent power flows from bus  $i$  to bus  $j$ , respectively. The parameters  $G_{ij}$  and  $B_{ij}$  are the real and imaginary parts of the  $(i,j)$ th entry of the nodal admittance matrix. The coefficients  $a_i$ ,  $b_i$ , and  $c_i$  specify the generation cost. The parameters  $\underline{P}_i^g$  and  $\bar{P}_i^g$  denote the active-power limits, while  $\underline{Q}_i^g$  and  $\bar{Q}_i^g$  denote the reactive-power limits. The parameters  $\underline{V}_i$  and  $\bar{V}_i$  are the voltage-magnitude limits, and  $\underline{\theta}_{ij}$  and  $\bar{\theta}_{ij}$  are the phase-angle-difference limits. The parameter  $\bar{S}_{ij}$  is the apparent-power thermal limit. Furthermore,  $R_i^{\text{up}}$  and  $R_i^{\text{down}}$  are the ramp-up and ramp-down limits, respectively, and the phase angle at reference bus  $r_0$  is fixed at zero.

### III. METHODOLOGY

Directly solving the nonconvex SMPC–ACOPF problem in (1) becomes computationally demanding as the prediction horizon and the number of uncertainty scenarios increase. To enable efficient learning-based solution, the proposed framework consists of three main components. First, the future scenario set is encoded into a compact permutation-invariant representation whose dimension is independent of the number of scenarios. Second, a conditional stochastic neural generator (CSNG) maps this representation and latent variables to multiple candidate dispatch trajectories, improving the chance of identifying feasible and low-cost solutions. Third, a pre-trained differentiable PF surrogate recovers the corresponding scenario-dependent network states, avoiding repeated nonlinear PF solutions during training. Together, these components enable efficient end-to-end training while accounting for both feasibility and economic performance.

#### A. Conditional Stochastic Neural Generator

Let  $C_{obj}(\cdot)$  denote the objective function in (1). The SMPC problem can then be compactly formulated as follows:

$$\min_{\mathbf{y}} C_{obj}(\mathbf{y}) \quad (2a)$$

$$\text{s.t. } \mathbf{h}(\mathbf{y}; \xi_s) = \mathbf{0}, \quad \forall s \in \mathcal{N}_s \quad (2b)$$

$$\mathbf{g}(\mathbf{y}; \xi_s) \leq \mathbf{0}, \quad \forall s \in \mathcal{N}_s \quad (2c)$$

where  $\mathbf{y}$  denotes the decision variables, and  $\xi_s$  denotes the load uncertainty trajectory associated with scenario  $s$ . The function  $\mathbf{h}(\cdot)$  collects the power balance equations and branch-flow equations, while  $\mathbf{g}(\cdot)$  collects the corresponding generator-output limits, voltage limits, phase-angle limits, transmission-line thermal limits, and ramping constraints.

1) *Motivation for Generative Modeling:* Most existing learning-based optimization methods employ a deterministic neural network as a surrogate for solving (2). However, such deterministic neural solvers typically provide only one candidate solution for each problem instance. For the considered SMPC–ACOPF problem, the nonlinear AC power-flow (PF) equations and operational limits define a highly nonconvex feasible region. Consequently, even a small prediction error may lead to constraint violations. Once the single prediction is infeasible, no alternative candidate is available for the same instance. Therefore, good average prediction accuracy does not necessarily translate into a high solution-recovery rate when strict feasibility is required. This observation motivates the use of a conditional stochastic neural generator (CSNG). Instead of producing a single deterministic solution, the CSNG  $G_\sigma(\cdot)$  parameterized by  $\sigma$  introduces a latent random variable  $\mathbf{z}$  and conceptually represents

$$\mathbf{y} = G_\sigma(\xi, \mathbf{z}), \quad \mathbf{z} \sim p_{\mathbf{z}}, \quad (3)$$

where  $p_{\mathbf{z}} = \mathcal{N}(\mathbf{0}, \mathbf{I})$  is the latent distribution. For a fixed SMPC instance, different realizations of  $\mathbf{z}$  produce different candidate dispatch trajectories.

Let  $\mathcal{U}_\xi^{\text{fea}}$  denote the set of dispatch trajectories whose corresponding AC PF realizations satisfy all physical and

operational constraints. The feasibility probability of a single generated candidate is defined as

$$p_{\text{fea}}(\xi; \theta) = \Pr_{\mathbf{z} \sim p_{\mathbf{z}}} [G_\sigma(\xi, \mathbf{z}) \in \mathcal{U}_\xi^{\text{fea}}]. \quad (4)$$

For a given SMPC instance,  $K$  independent latent samples are drawn as

$$\mathbf{z}^{(k)} \stackrel{\text{i.i.d.}}{\sim} p_{\mathbf{z}}, \quad k = 1, \dots, K, \quad (5)$$

and the corresponding candidate dispatch trajectories are

$$\mathbf{y}^{(k)} = G_\sigma(\xi, \mathbf{z}^{(k)}), \quad k = 1, \dots, K. \quad (6)$$

Under independent latent sampling, the probability that at least one candidate belongs to  $\mathcal{U}_\xi^{\text{fea}}$  is

$$P_{\text{hit}}^{(K)}(\xi; \theta) = 1 - [1 - p_{\text{fea}}(\xi; \theta)]^K. \quad (7)$$

Equation (7) highlights the main advantage of generative modeling in the present setting: multiple candidates provide multiple opportunities to recover a feasible solution for the same SMPC instance. At inference, the generated candidates are first screened according to feasibility, and the lowest-cost candidate is subsequently selected from the feasible subset.

2) *Generator Architecture:* The representation in (6) is conceptual. A direct implementation would require concatenating all future scenario realizations  $\{\xi_s\}_{s \in \mathcal{N}_s}$  as the generator input. Since each  $\xi_s$  contains the uncertain renewable generation and load over the entire recourse horizon, the resulting input dimension would grow linearly with the number of scenarios  $|\mathcal{N}_s|$ . To obtain a compact and fixed-dimensional representation of the scenario set, we apply empirical pooling [6] over the scenario dimension. Specifically, element-wise minimum, mean, and maximum pooling are performed as

$$\xi_{\text{pool}} = \left( \min_{s \in \mathcal{N}_s} \xi_s, \frac{1}{|\mathcal{N}_s|} \sum_{s \in \mathcal{N}_s} \xi_s, \max_{s \in \mathcal{N}_s} \xi_s \right), \quad (8)$$

where the minimum and maximum operations are evaluated component-wise. The pooled representation captures the empirical range and central tendency of the future uncertainty realizations while maintaining a dimension independent of  $|\mathcal{N}_s|$ . To capture the temporal dependence of multi-period dispatch decisions induced by intertemporal constraints, the stochastic generator is implemented using a non-causal temporal convolutional network (TCN). This architecture allows the decision at each time period to exploit uncertainty information over the entire prediction horizon while preserving the temporal structure of the uncertainty trajectories. Accordingly, the pooled representation in (8) is arranged as the temporal sequence

$$\xi_{\text{pool}} = [(\xi_1^{\text{pool}})^\top \quad \dots \quad (\xi_T^{\text{pool}})^\top]^\top \in \mathbb{R}^{T \times 3d_\xi}, \quad (9)$$

where  $\xi_t^{\text{pool}} \in \mathbb{R}^{3d_\xi}$  denotes the minimum–mean–maximum pooled uncertainty feature at time  $t$ , and  $d_\xi$  denotes the dimension of the uncertain quantities at each time period.

For each candidate trajectory, a latent vector  $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  is sampled and shared across all time periods. This trajectory-level latent variable enables different realizations of  $\mathbf{z}$  to represent distinct dispatch trajectories rather than independent

perturbations at individual time periods. The latent vector is first embedded and then concatenated with the pooled uncertainty feature at each time step:

$$\mathbf{h}_t^{(0)} = \phi_{\text{in}} \left( \xi_t^{\text{pool}}, \phi_z(\mathbf{z}) \right), \quad t \in \mathcal{T}, \quad (10)$$

where  $\phi_z(\cdot)$  denotes the learnable latent embedding,  $\phi_{\text{in}}(\cdot)$  denotes the input embedding, and  $\mathbf{h}_t^{(0)}$  is the resulting input feature of the TCN at time  $t$ .

The embedded sequence is then processed by stacked non-causal dilated convolutional blocks. At the  $\ell$ th layer, the convolutional feature at time  $t$  is computed as

$$\mathbf{c}_t^{(\ell)} = \sum_{q=-m}^m \mathbf{W}_q^{(\ell)} \mathbf{h}_{t+qd_\ell}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \quad k_c = 2m + 1, \quad (11)$$

where  $k_c$  is the convolution kernel size,  $m = (k_c - 1)/2$  specifies the temporal neighborhood on each side of  $t$ ,  $d_\ell$  is the dilation factor of layer  $\ell$ , and  $\mathbf{W}_q^{(\ell)}$  and  $\mathbf{b}^{(\ell)}$  denote the convolution weights and bias, respectively. Here,  $\mathbf{h}_t^{(\ell-1)}$  denotes the input feature to the  $\ell$ th block, while  $\mathbf{c}_t^{(\ell)}$  is subsequently passed through the nonlinear and residual operations of the block to obtain  $\mathbf{h}_t^{(\ell)}$ . Symmetric padding is employed to preserve the sequence length.

Because (11) incorporates information from both sides of time  $t$ , stacking dilated convolutional blocks progressively enlarges the temporal receptive field. The dilation configuration is selected such that the final receptive field covers the entire prediction horizon. Denoting the resulting non-causal TCN by  $\text{NCTCN}_{\sigma_h}(\cdot)$ , where  $\sigma_h$  collects its trainable parameters, the horizon-wide representation is written compactly as

$$\mathbf{H} = \text{NCTCN}_{\sigma_h}(\xi_{\text{pool}}, \mathbf{z}), \quad (12)$$

where  $\mathbf{H}$  denotes the resulting temporal feature sequence that jointly encodes the pooled uncertainty information and the latent realization.

On the output side, directly generating the complete operating point  $\mathbf{y}$  would require the CSNG to produce samples satisfying the nonlinear PF equality constraints. To avoid imposing this burden on the generator, the variables are partitioned as

$$\mathbf{y} = \begin{bmatrix} \mathbf{u} \\ \mathbf{w} \end{bmatrix}, \quad (13)$$

where  $\mathbf{u}$  contains the free decision variables and  $\mathbf{w}$  contains the dependent variables determined by the AC PF equations. Specifically, the free-variable vector consists of the active-power set points at PV buses and the voltage-magnitude set points at PV buses and the reference bus, i.e.,  $\mathbf{u} = (\mathbf{P}_{\text{PV}}^g, \mathbf{V}_{\text{PV}}, \mathbf{V}_{r_0})$ . The remaining system states are recovered from the AC PF equations for the given uncertainty realization. Accordingly, the free variables are generated as

$$\mathbf{u} = G_\sigma^u(\xi_{\text{pool}}, \mathbf{z}) = \phi_{\text{out}, \sigma_o}(\text{NCTCN}_{\sigma_h}(\xi_{\text{pool}}, \mathbf{z})). \quad (14)$$

where  $\phi_{\text{out}, \sigma_o}(\cdot)$  denotes the output mapping that transforms the temporal feature sequence  $\mathbf{H}$  into the free-variable trajectory, and  $\sigma = \{\sigma_h, \sigma_o\}$  collects the trainable parameters of the NCTCN backbone and the output mapping.

Given  $\mathbf{u}$  and the full scenario realization  $\xi$ , the corresponding operating point is completed through

$$\mathbf{y} = \begin{bmatrix} \mathbf{u} \\ \mathbf{w} \end{bmatrix} = \begin{bmatrix} G_\sigma^u(\xi_{\text{pool}}, \mathbf{z}) \\ \psi_{\text{pf}}(G_\sigma^u(\xi_{\text{pool}}, \mathbf{z}); \xi) \end{bmatrix}, \quad (15)$$

where  $\psi_{\text{pf}}(\cdot)$  denotes the equality-completion mapping induced by the AC PF equations.

The structure of  $\mathbf{u}$  also allows several operational constraints to be enforced directly at the generator output. Let

$$\tilde{\mathbf{u}} = (\tilde{\mathbf{P}}_{\text{PV}}^g, \tilde{\mathbf{V}}_{\text{PV}}, \tilde{\mathbf{V}}_{r_0}) = \tilde{G}_\sigma^u(\xi_{\text{pool}}, \mathbf{z}) \quad (16)$$

denote the raw output of the stochastic neural generator.

3) *Constraint-Aware Projection*: Bounded decision variables are often parameterized using sigmoid mappings to enforce box constraints. However, the sigmoid gradient diminishes near the bounds, which may weaken the optimization signal when constraints become active. This is particularly undesirable in OPF, where economically optimal solutions often lie on the boundary of the feasible region. We therefore adopt the following constraint-aware projection:

$$\mathcal{P}(\tilde{\mathbf{u}}) = \text{Proj}_{[0,1]}(\tilde{\mathbf{u}}) = \min\{1, \max\{0, \tilde{\mathbf{u}}\}\}. \quad (17)$$

Accordingly, the voltage components are mapped to their admissible ranges by

$$V_{i,t} = \underline{V}_i + (\bar{V}_i - \underline{V}_i) \mathcal{P}(\tilde{V}_{i,t}). \quad (18)$$

Similarly, the active-power components are parameterized recursively to enforce both generation and ramping limits. Given  $P_{i,t-1}^g$ , the admissible interval at time  $t$  is

$$\underline{P}_{i,t}^{\text{adm}} = \max\{\underline{P}_i^g, P_{i,t-1}^g - R_i^{\text{down}}\}, \quad (19a)$$

$$\bar{P}_{i,t}^{\text{adm}} = \min\{\bar{P}_i^g, P_{i,t-1}^g + R_i^{\text{up}}\}. \quad (19b)$$

The corresponding active-power dispatch is then obtained as

$$P_{i,t}^g = \underline{P}_{i,t}^{\text{adm}} + (\bar{P}_{i,t}^{\text{adm}} - \underline{P}_{i,t}^{\text{adm}}) \mathcal{P}(\tilde{P}_{i,t}^g). \quad (20)$$

Collecting the above transformations gives

$$\mathbf{u} = \mathcal{F}_{\text{adm}}(\tilde{\mathbf{u}}), \quad G_\sigma^u = \mathcal{F}_{\text{adm}} \circ \tilde{G}_\sigma^u. \quad (21)$$

where  $\mathcal{F}_{\text{adm}}(\cdot)$  denotes the deterministic constraint-preserving projection mapping. Consequently, the CSNG only needs to account for the remaining constraints associated with the dependent variables recovered through  $\psi_{\text{pf}}(\cdot)$ .

Although the projection in (17) allows the generator to attain the variable bounds exactly, conventional clipping blocks gradient propagation once the raw output enters a saturated region. Specifically, let

$$\mathbf{u} = \text{Proj}_{[0,1]}(\tilde{\mathbf{u}}), \quad g_u = \frac{\partial \mathcal{L}}{\partial \mathbf{u}}, \quad (22)$$

where  $g_u$  denotes the upstream gradient of the current training objective  $\mathcal{L}$ . Under the standard clipping operator, the gradient with respect to the raw output is

$$\left. \frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{u}}} \right|_{\text{clip}} = \frac{\partial \mathcal{L}}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \tilde{\mathbf{u}}} = \begin{cases} g_u, & 0 < \tilde{\mathbf{u}} < 1, \\ 0, & \tilde{\mathbf{u}} < 0 \text{ or } \tilde{\mathbf{u}} > 1, \end{cases} \quad (23)$$

Hence, once  $\tilde{\mathbf{u}}$  enters the saturated region, informative gradient signals are blocked during backpropagation. To address this issue, we customize its backward propagation as follows:

$$\left. \frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{u}}} \right|_{\text{clip}} = \begin{cases} g_{\mathbf{u}}, & 0 < \mathbf{u} < 1, \\ g_{\mathbf{u}}, & \mathbf{u} = 0 \text{ and } g_{\mathbf{u}} < 0, \\ g_{\mathbf{u}}, & \mathbf{u} = 1 \text{ and } g_{\mathbf{u}} > 0, \\ 0, & \text{otherwise.} \end{cases} \quad (24)$$

The rule has a simple interpretation. At the lower bound  $\mathbf{u} = 0$ ,  $g_{\mathbf{u}} < 0$  is retained because the corresponding gradient-descent step increases  $\mathbf{u}$ , whereas  $g_{\mathbf{u}} > 0$  is blocked because it would move the variable further outside the feasible interval. The upper-bound case is symmetric. Thus, the customized rule preserves the exact forward projection while allowing a saturated variable to leave the boundary whenever the training objective favors an interior solution.

### B. Feasibility-Aware Self-Supervised End-to-End Training

The CSNG is trained to generate a diverse set of candidate dispatch trajectories concentrated in feasible regions with low operating costs. A conventional supervised-learning approach would require a large number of labeled samples obtained by repeatedly solving the original SMPC-ACOPF problem in (1) for different uncertainty instances, which would incur substantial computational cost. We therefore train the CSNG in a self-supervised manner by directly evaluating the operational constraints and objective function of the SMPC model on the generated candidates. For each training instance,  $K_{\text{tr}}$  candidate dispatch trajectories are sampled as

$$\mathbf{u}^{(k)} = G_{\sigma}^u \left( \xi_{\text{pool}}, \mathbf{z}^{(k)} \right), \quad \mathbf{z}^{(k)} \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad k = 1, \dots, K_{\text{tr}}. \quad (25)$$

Each free-variable candidate is completed to the full operating point  $\mathbf{y}^{(k)}$  through (15). The resulting inequality-constraint residuals and operating costs are then used to train CSNG.

1) *Constraint Violation*: A direct average over all inequality constraints may dilute the gradients associated with a small number of critical violations, particularly in large-scale SMPC problems involving many constraints, time periods, and uncertainty scenarios. To address this issue, we adopt a hierarchical mean-CVaR aggregation that preserves dense gradient information while emphasizing the most critical violations.

Let  $\mathcal{C}$  denote the set of inequality-constraint families that remain after applying the admissible-output mapping  $\mathcal{F}_{\text{adm}}$ , including generator reactive-power limits, reference-bus power limits, P-Q bus voltage limits, phase-angle-difference limits, and transmission-line thermal limits. For candidate  $k$ , scenario  $s$ , time period  $t$ , and constraint family  $c \in \mathcal{C}$ , define the normalized positive residual vector of (2c) as

$$\mathbf{r}_{k,s,t}^{(c)} = \left[ \mathbf{D}_c \mathbf{g}_{s,t}^{(c)} \left( \mathbf{y}^{(k)} \right) \right]_+, \quad (26)$$

where  $\mathbf{D}_c$  is a diagonal scaling matrix that normalizes the numerical scales of different physical constraints. For a non-negative vector  $\mathbf{r}$ , let  $\text{CVaR}_{\rho_c}(\mathbf{r})$  denote the empirical mean

of its largest  $\rho_c$  fraction of elements. The violation associated with constraint family  $c$  is defined as

$$\mathcal{V}_{k,s,t}^{(c)} = \alpha_c \frac{\|\mathbf{r}_{k,s,t}^{(c)}\|_1}{m_c} + (1 - \alpha_c) \text{CVaR}_{\rho_c} \left( \mathbf{r}_{k,s,t}^{(c)} \right), \quad (27)$$

where  $m_c$  is the number of scalar inequalities in family  $c$ , and  $\alpha_c \in (0, 1)$  controls the tradeoff between dense mean-violation feedback and emphasis on the upper-tail violations.

The resulting feasibility loss over the generated candidate dispatch trajectories is

$$\mathcal{L}_{\text{fea}} = \sum_{k=1}^{K_{\text{tr}}} \sum_{s \in \mathcal{N}_s} \sum_{t \in \mathcal{T}} \sum_{c \in \mathcal{C}} \mathcal{V}_{k,s,t}^{(c)}. \quad (28)$$

2) *Diversity Preservation*: Generating multiple candidates is useful only if different latent samples can lead to distinct operating points. Excessive concentration of the generated candidates in a narrow region would reduce the benefit of stochastic sampling. We therefore introduce a diversity term among candidates that have small constraint violations. The free-variable vector contains quantities with different physical ranges. To obtain a meaningful distance measure, each component is normalized according to its operating range:

$$\hat{u}^{(k)} = \frac{u^{(k)} - u^c}{\Delta u}, \quad u^c = \frac{\bar{u} + \underline{u}}{2}, \quad \Delta u = \frac{\bar{u} - \underline{u}}{2}, \quad (29)$$

where  $\underline{u}$  and  $\bar{u}$  denote the corresponding lower and upper admissible limits.

For feasibility-aware distribution shaping, each candidate is assigned a soft feasibility score

$$s_k = \exp \left( - \frac{\sum_{s \in \mathcal{N}_s} \sum_{t \in \mathcal{T}} \sum_{c \in \mathcal{C}} \mathcal{V}_{k,s,t}^{(c)}}{\tau_f} \right), \quad 0 < s_k \leq 1. \quad (30)$$

where  $\tau_f > 0$  determines the sensitivity to constraint violations. Candidates satisfying or closely approaching the operational constraints have  $s_k$  close to one, whereas candidates with large violations have small scores. The score is used as a fixed feasibility-dependent weight in the distribution-shaping terms below.

To prevent different latent samples from collapsing onto nearby operating points, we extend the kernel-based repulsive regularization of [7] by incorporating the proposed soft feasibility score. Specifically, pairwise similarity is measured through a Gaussian kernel defined in the normalized free-variable space.

$$\mathcal{L}_{\text{div}} = \frac{\sum_{i < j} \text{sg}(s_i s_j) \exp \left( - \frac{\|\hat{\mathbf{u}}^{(i)} - \hat{\mathbf{u}}^{(j)}\|_2^2}{2\sigma_d^2} \right)}{\sum_{i < j} \text{sg}(s_i s_j) + \varepsilon}, \quad (31)$$

where  $\sigma_d > 0$  specifies the characteristic separation between candidates,  $\varepsilon > 0$  is a small constant for numerical stability, and  $\text{sg}(\cdot)$  denotes the stop-gradient operator. The Gaussian kernel is large for two nearby operating points and decreases as their distance increases. Therefore, minimizing (31) discourages near-duplicate candidates.



The factors  $s_i s_j$  make this separation mainly act on candidate pairs that are close to satisfying the operating constraints. For example, if  $s_1 = 0.95$ ,  $s_2 = 0.90$ , and  $s_3 = 0.02$ , the pairwise weights are  $s_1 s_2 = 0.855$  and  $s_1 s_3 = 0.019$ , respectively. Consequently, two near-feasible candidates are encouraged to represent different operating points, whereas a strongly infeasible candidate receives little diversity-related force. The stop-gradient operation makes  $s_i s_j$  act only as a fixed weight. Otherwise, the diversity loss could also be reduced by decreasing  $s_k$ , which corresponds to increasing the constraint violation of a candidate. With stop-gradient, the diversity term changes the candidate locations without modifying the feasibility weighting mechanism.

3) *Economic Shaping*: The economic objective should guide the generated candidates toward low-cost and feasible operating regions. Directly minimizing generation cost may favor infeasible solutions with artificially low costs. Moreover, since the lowest-cost feasible candidate is selected at inference, additional emphasis should be placed on the most economically competitive candidates. We therefore incorporate the soft feasibility score  $s_k$  in (30) and adopt a mean-best- $K_b$  aggregation:

$$\begin{aligned} \mathcal{L}_{\text{eco}} = & \alpha_{\text{eco}} \frac{1}{K_{\text{tr}}} \sum_{k=1}^{K_{\text{tr}}} C_{\text{obj}}(\mathbf{y}^{(k)}) [1 + \gamma_{\text{eco}} (1 - s_k)] \\ & + (1 - \alpha_{\text{eco}}) \frac{1}{K_b} \sum_{k \in \mathcal{I}_b} C_{\text{obj}}(\mathbf{y}^{(k)}) [1 + \gamma_{\text{eco}} (1 - s_k)], \end{aligned} \quad (32)$$

where  $\gamma_{\text{eco}} > 0$  controls the feasibility-dependent amplification and  $\alpha_{\text{eco}} \in (0, 1)$  balances the contributions of the full candidate set and the best- $K_b$  subset. The index set  $\mathcal{I}_b$  contains the  $K_b$  candidates with the smallest feasibility-adjusted objective values  $C_{\text{obj}}(\mathbf{y}^{(k)})[1 + \gamma_{\text{eco}}(1 - s_k)]$ . As  $s_k$  decreases with increasing constraint violation, infeasible candidates receive a larger economic penalty. The first term improves the overall economic quality of the generated distribution, while the second focuses optimization on the most competitive candidates likely to be selected at inference.

The feasibility loss reduces constraint violations, while the diversity loss discourages near-duplicate solutions. The economic shaping further promotes low-cost candidates. Together, these terms guide the stochastic generator toward diverse, feasible, and economically competitive dispatch trajectories. The complete CSNG training objective is therefore given by

$$\mathcal{L}_{\text{train}} = \lambda_{\text{fea}} \mathcal{L}_{\text{fea}} + \lambda_{\text{div}} \mathcal{L}_{\text{div}} + \lambda_{\text{eco}} \mathcal{L}_{\text{eco}}, \quad (33)$$

where  $\lambda_{\text{fea}}$ ,  $\lambda_{\text{div}}$ , and  $\lambda_{\text{eco}}$  balance the numerical scales and relative contributions of the three training terms.

To improve training stability, a two-stage training strategy is adopted. During the first stage, the CSNG is trained using  $\mathcal{L}_{\text{fea}}$  and  $\mathcal{L}_{\text{div}}$  to establish a feasible and non-collapsed candidate distribution. During the second stage,  $\mathcal{L}_{\text{eco}}$  is activated while the feasibility and diversity terms remain active, thereby shifting the generated distribution toward lower-cost regions without sacrificing feasibility or diversity. The complete training procedure is summarized in Algorithm 1.

---

**Algorithm 1:** Two-Stage Self-Supervised End-to-End Training of the Conditional Stochastic Generator

---

**Input:** training set  $\mathcal{D}$ ; CSNG parameters  $\sigma$ ; number of generated candidates  $K_{\text{tr}}$ ; stage boundary  $E_1$ ; total epochs  $E$ ; learning rate  $\eta$

**Output:** trained CSNG parameters  $\sigma$

```

1 Initialize the CSNG parameters  $\sigma$ ;
2 for  $e = 1 : E$  do
3   Sample a mini-batch of SMPC uncertainty
   instances  $\xi$  from  $\mathcal{D}$ ;
4   foreach  $\xi$  in the mini-batch do
5     Construct the pooled representation  $\xi_{\text{pool}}$ ;
6     Sample  $\mathbf{z}^{(k)} \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(\mathbf{0}, \mathbf{I})$  for  $k = 1, \dots, K_{\text{tr}}$ ;
7     Generate candidate dispatch trajectories
        $\mathbf{u}^{(k)} = G_{\sigma}^u(\xi_{\text{pool}}, \mathbf{z}^{(k)})$ ;
8     Recover the corresponding operating points
        $\mathbf{y}^{(k)}$  through (15);
9     Compute  $\mathcal{L}_{\text{fea}}$  using (26)–(28);
10    Compute  $\mathcal{L}_{\text{div}}$  using (29)–(31);
11    if  $e \leq E_1$  then
12      Set  $\mathcal{L} = \lambda_{\text{fea}} \mathcal{L}_{\text{fea}} + \lambda_{\text{div}} \mathcal{L}_{\text{div}}$ ;
13    else
14      Compute  $\mathcal{L}_{\text{eco}}$  using (32);
15      Set  $\mathcal{L} = \lambda_{\text{fea}} \mathcal{L}_{\text{fea}} + \lambda_{\text{div}} \mathcal{L}_{\text{div}} + \lambda_{\text{eco}} \mathcal{L}_{\text{eco}}$ ;
16    end
17  end
18  Update  $\sigma \leftarrow \sigma - \eta \nabla_{\sigma} \mathcal{L}$ ;
19 end

```

---

### C. Differentiable Equality Completion Surrogate

All operations involved in Algorithm 1 are amenable to standard automatic differentiation except for the equality-completion mapping in (15). For a given set of free variables and an uncertainty realization, this mapping requires solving a nonlinear AC PF problem to recover the remaining dependent variables. Although the required sensitivities can in principle be obtained through implicit differentiation [8], such an approach is undesirable for the large-scale scenario-based training problem considered here. In particular, each generated candidate requires a converged PF solution for every scenario and time period, followed by Jacobian-based linear solves during backpropagation. Consequently, both the computational and memory costs grow rapidly with the numbers of scenarios and generated candidates.

To remove the nonlinear PF solver from the end-to-end CSNG training loop, we introduce a differentiable equality completion surrogate (DECS). The key idea is to learn only the implicit mapping from the standard PF specifications to the minimum set of unknown voltage states. Once these states are predicted, all remaining dependent quantities, including the reactive-power outputs at P-V buses, the active and reactive power injections at the reference bus, and the branch power flows, are recovered analytically through the AC network equations. Consequently, the learned component is restricted to the genuinely implicit part of the PF mapping, whereas

the remaining physical relationships are retained exactly as deterministic differentiable operations.

1) *PF Specification and Minimal-State Mapping*: Let  $\mathcal{N}_{PV}$  and  $\mathcal{N}_{PQ}$  denote the sets of P-V and P-Q buses, respectively, where the reference bus  $r_0$  is excluded from  $\mathcal{N}_{PV}$ . Let  $\rho^{PV}$  and  $\rho^{PQ}$  denote the PF specifications associated with the P-V and P-Q buses, respectively, and let  $\rho$  denote the complete PF specification vector. Specifically,

$$\rho^{PV} = (\mathbf{p}_{PV}^{sp}, \mathbf{V}_{PV}), \quad (34a)$$

$$\rho^{PQ} = (\mathbf{p}_{PQ}^{sp}, \mathbf{q}_{PQ}^{sp}), \quad (34b)$$

$$\rho = (\rho^{PV}, V_{r_0}, \rho^{PQ}). \quad (34c)$$

where  $\mathbf{p}_{PV}^{sp}$  denotes the vector of net active-power injections at the P-V buses,  $\mathbf{V}_{PV}$  denotes the corresponding voltage magnitudes, and  $\mathbf{p}_{PQ}^{sp}$  and  $\mathbf{q}_{PQ}^{sp}$  denote the vectors of net active- and reactive-power injections at the P-Q buses, respectively. The quantity  $V_{r_0}$  denotes the voltage magnitude at the reference bus, whose phase angle is fixed as  $\theta_{r_0} = 0$ .

In the proposed SMPC framework,  $\rho$  is determined jointly by the free variables  $\mathbf{u}$  and the uncertainty realization  $\xi$ , which is compactly expressed as  $\rho = \rho(\mathbf{u}; \xi)$ . Given  $\rho$ , let  $\chi$  denote the minimal set of unknown PF states,

$$\chi = (\theta_{PV}, \theta_{PQ}, \mathbf{V}_{PQ}), \quad (35)$$

The AC PF equations implicitly define the mapping  $\chi = \phi_{pf}(\rho)$ . The proposed DECS replaces the repeated numerical evaluation of  $\phi_{pf}(\cdot)$  with a differentiable surrogate  $\Phi_\omega(\cdot)$ :

$$\hat{\chi} = \Phi_\omega(\rho), \quad (36)$$

where  $\omega$  denotes the trainable parameters of the surrogate.

2) *Physics-Guided Training*: The DECS is pretrained offline and is kept fixed during the subsequent self-supervised training of the CSNG. The surrogate training set is constructed by sampling PF specifications over the operating region relevant to the SMPC problem. For each sampled PF specification  $\rho^{(n)}$ , an AC PF solver is executed once offline to obtain the corresponding solution  $\chi^{(n)}$ . The resulting data set is

$$\mathcal{D}_{pf} = \left\{ \left( \rho^{(n)}, \chi^{(n)} \right) \right\}_{n=1}^{N_{pf}}. \quad (37)$$

A purely data-driven supervised loss can be defined as

$$\mathcal{L}_{sup} = \frac{1}{N_{pf}} \sum_{n=1}^{N_{pf}} \|\hat{\chi}^{(n)} - \chi^{(n)}\|_2^2. \quad (38)$$

However, a purely data-driven supervised loss does not explicitly capture the underlying physical relationships governed by the PF equations, which may limit the generalization capability of the model. To enhance generalization, we therefore augment the supervised objective with an explicit AC-physics loss. For compactness, define the AC nodal-injection functions in (1b) and (1c) as  $\mathcal{P}_i(\mathbf{V}, \theta)$  and  $\mathcal{Q}_i(\mathbf{V}, \theta)$ . For a predicted state  $\hat{\chi}^{(n)}$ , the complete voltage vector is assembled using the specified P-V/reference-bus voltage magnitudes and the predicted P-Q voltage magnitudes, whereas the reference angle is fixed at zero. Denote the resulting voltage state by  $(\hat{\mathbf{V}}^{(n)}, \hat{\theta}^{(n)})$ . The active-power residuals at all nonreference

buses and the reactive-power residuals at the P-Q buses are then

$$\mathbf{r}_P^{(n)} = \mathcal{P}_{\mathcal{N}_{PV} \cup \mathcal{N}_{PQ}} \left( \hat{\mathbf{V}}^{(n)}, \hat{\theta}^{(n)} \right) - \mathbf{p}_{\mathcal{N}_{PV} \cup \mathcal{N}_{PQ}}^{sp, (n)}, \quad (39a)$$

$$\mathbf{r}_Q^{(n)} = \mathcal{Q}_{\mathcal{N}_{PQ}} \left( \hat{\mathbf{V}}^{(n)}, \hat{\theta}^{(n)} \right) - \mathbf{q}_{\mathcal{N}_{PQ}}^{sp, (n)}. \quad (39b)$$

No reactive-power residual is imposed at a P-V bus, and no active- or reactive-power residual is imposed at the reference bus, because these quantities are dependent PF outputs rather than specified inputs. The physics-guided loss and the resulting DECS training objective are defined as

$$\mathcal{L}_{phy} = \frac{1}{N_{pf}} \sum_{n=1}^{N_{pf}} \left[ \|\mathbf{r}_P^{(n)}\|_2^2 + \|\mathbf{r}_Q^{(n)}\|_2^2 \right], \quad (40a)$$

$$\omega^* = \arg \min_{\omega} (\mathcal{L}_{sup} + \lambda_{phy} \mathcal{L}_{phy}), \quad (40b)$$

where  $\lambda_{phy} > 0$  controls the relative emphasis placed on physical consistency.

3) *Exact Differentiable AC Reconstruction*: The neural network in (36) predicts only the minimal voltage-state vector. All remaining PF-dependent quantities are subsequently recovered from the AC equations through a parameter-free reconstruction layer. For each scenario  $s$  and time period  $t$ , the full voltage magnitudes and phase angles are first assembled as

$$\hat{V}_{s,i,t} = \begin{cases} V_{s,i,t}, & i \in \mathcal{N}_{PV} \cup \{r_0\}, \\ [\hat{\chi}_{s,t}]_{V_i}, & i \in \mathcal{N}_{PQ}, \end{cases} \quad (41a)$$

$$\hat{\theta}_{s,i,t} = \begin{cases} 0, & i = r_0, \\ [\hat{\chi}_{s,t}]_{\theta_i}, & i \in \mathcal{N}_{PV} \cup \mathcal{N}_{PQ}. \end{cases} \quad (41b)$$

Using these voltages, the active- and reactive-power outputs of each conventional generator is reconstructed as

$$\hat{P}_{s,i,t}^g = \mathcal{P}_i \left( \hat{\mathbf{V}}_{s,t}, \hat{\theta}_{s,t} \right) - P_{s,i,t}^r + P_{s,i,t}^d, \quad (42a)$$

$$\hat{Q}_{s,i,t}^g = \mathcal{Q}_i \left( \hat{\mathbf{V}}_{s,t}, \hat{\theta}_{s,t} \right) - Q_{s,i,t}^r + Q_{s,i,t}^d. \quad (42b)$$

The branch active- and reactive-power flows  $\hat{P}_{s,ij,t}$  and  $\hat{Q}_{s,ij,t}$  are then evaluated directly using (1d) and (1e), respectively, after substituting  $(\hat{\mathbf{V}}_{s,t}, \hat{\theta}_{s,t})$ . The squared apparent-power magnitude can be evaluated without a square-root operation as

$$|\hat{S}_{s,ij,t}|^2 = \hat{P}_{s,ij,t}^2 + \hat{Q}_{s,ij,t}^2. \quad (43)$$

Collecting the predicted voltage states and all analytically reconstructed quantities gives

$$\hat{\mathbf{w}}_{s,t} = \mathcal{R}_{AC} \left( \rho_{s,t}, \Phi_{\omega^*}(\rho_{s,t}); \xi_{s,t} \right), \quad (44)$$

where  $\mathcal{R}_{AC}(\cdot)$  denotes the AC reconstruction operator consisting only of the algebraic operations in (41)–(43). The resulting DECS mapping can therefore be expressed as

$$\rho_{s,t} = \rho(\mathbf{u}_t; \xi_{s,t}), \quad (45a)$$

$$\hat{\chi}_{s,t} = \Phi_{\omega^*}(\rho_{s,t}), \quad (45b)$$

$$\hat{\psi}_{pf}(\mathbf{u}_t; \xi_{s,t}) = \mathcal{R}_{AC}(\rho_{s,t}, \hat{\chi}_{s,t}; \xi_{s,t}). \quad (45c)$$

TABLE I  
METHOD COMPARISON ON THE IEEE 14-BUS SYSTEM

| Method | Objective Value | Feasibility Rate | Computation Time |
|--------|-----------------|------------------|------------------|
| IPOPT  |                 |                  |                  |
| D-NN   |                 |                  |                  |
| S-CSNG |                 |                  |                  |
| D-CSNG |                 |                  |                  |
| CSNG   |                 |                  |                  |

Thus, during CSNG training, the original non-differentiable equality-completion operation in (15) is replaced by the following differentiable operator:

$$\hat{\mathbf{y}}^{(k)} = \begin{bmatrix} \mathbf{u}^{(k)} \\ \hat{\psi}_{\text{pf}}(\mathbf{u}^{(k)}; \boldsymbol{\xi}) \end{bmatrix}. \quad (46)$$

#### IV. CASE STUDY

To evaluate the effectiveness of the proposed method, numerical experiments are conducted on the IEEE 14-bus, IEEE 118-bus, and PEGASE 1354-bus systems [9]. The scheduling horizon contains 16 time periods, with 20 uncertainty scenarios considered for each instance. The uncertain quantities are sampled within  $\pm 15\%$  of their nominal values. The datasets are divided into training, validation, and test sets with a ratio of 8:1:1. The training set is used for parameter optimization, the validation set for early stopping and overfitting control, and the test set for final performance evaluation. PF calculations are performed using Power Grid Model [10], which supports parallel PF computations, while the neural networks are implemented in PyTorch. All experiments are conducted on a laptop equipped with an Intel i9-14900HX CPU, an NVIDIA RTX 4060 GPU, and 32 GB of RAM.

The proposed method is compared with the following approaches: (i) **IPOPT**, which directly solves the nonconvex SMPC problem using IPOPT; (ii) **D-NN**, which replaces the stochastic generator with a deterministic neural network while retaining the same training framework; (iii) **S-CSNG**, which adopts the proposed stochastic generative architecture but uses sigmoid-based output parameterization; (iv) **WD-CSNG**, which removes the diversity-preservation term from the proposed training objective; and (v) **CSNG**, the complete proposed stochastic generative method. At inference, all candidates generated by CSNG are evaluated in parallel using Power Grid Model [10], and the feasible candidate with the lowest operating cost is selected as the final solution. All data, hyperparameter settings, and implementation source codes are available at <https://github.com/JieZhu6/RTN-DER-DFG>.

##### A. Case Study on the IEEE 14-Bus System

For the IEEE 14-bus system, 5,000 uncertainty scenarios are generated for training the CSNG, while 10,000 PF operating points are generated for training the DECS.

TABLE II  
METHOD COMPARISON ON THE IEEE 118-BUS SYSTEM

| Method | Objective Value | Feasibility Rate | Computation Time |
|--------|-----------------|------------------|------------------|
| IPOPT  |                 |                  |                  |
| D-NN   |                 |                  |                  |
| S-CSNG |                 |                  |                  |
| D-CSNG |                 |                  |                  |
| CSNG   |                 |                  |                  |

TABLE III  
METHOD COMPARISON ON THE PEGASE 1354-BUS SYSTEM

| Method | Objective Value | Feasibility Rate | Computation Time |
|--------|-----------------|------------------|------------------|
| IPOPT  |                 |                  |                  |
| D-NN   |                 |                  |                  |
| S-CSNG |                 |                  |                  |
| D-CSNG |                 |                  |                  |
| CSNG   |                 |                  |                  |

##### B. Case Study on the IEEE 118-bus System

##### C. Case Study on the PEGASE 1354-bus System

#### V. CONCLUSION

#### REFERENCES

- [1] M. Hoseinpour and V. Dvorkin, "Diffopf: Diffusion solver for optimal power flow," *Electric Power Systems Research*, vol. 262, p. 113621, 2027.
- [2] —, "Constrained diffusion models for synthesizing representative power flow datasets," *IEEE Transactions on Power Systems*, pp. 1–13, 2026.
- [3] H. Meng, X. Feng, Y. Li, C. Wang, K. Rajput, M. Schram, and H. Chen, "Solving constrained optimization problems as ode-based models using reinforcement learning," *Transactions on Machine Learning Research*, 2026.
- [4] E. Liang and M. Chen, "Generative learning for solving non-convex problem with multi-valued input-solution mapping," in *International Conference on Learning Representations*, 2024.
- [5] S. Ding, Y. Zhou, K. Hu, X. Yao, J. Yan, X. Tang, and Y. Shi, "Diffusion-based learning framework for constrained nonconvex optimization with weighted bootstrapped refinement," in *International Conference on Learning Representations*, 2026.
- [6] Z. Shao, J. Qin, and N. Yu, "Neural two-stage stochastic volt-var optimization for three-phase unbalanced distribution systems with network reconfiguration," *arXiv preprint arXiv:2510.23867*, 2025.
- [7] W. Wang, Y. Sun, and S. Halgamuge, "Improving mmd-gan training with repulsive loss function," in *International Conference on Learning Representations*, 2019.
- [8] P. Donti, D. Rolnick, and J. Z. Kolter, "Dc3: A learning method for optimization with hard constraints," in *International Conference on Learning Representations*, 2021.
- [9] "PGLib-OPF: Benchmarks for the Optimal Power Flow Problem," <https://github.com/power-grid-lib/pglib-opf>.
- [10] Y. Xiang et al., "PowerGridModel/power-grid-model." [Online]. Available: <https://github.com/PowerGridModel/power-grid-model>